目录

基于 CNN 的校园课堂智能考勤系统 — 详细设计文档 **1**
1. 用户认证模块详细设计 2
1.1 登录接口（POST /api/auth/login） 2
2. 学生档案模块详细设计 3
2.1 分页查询（GET /api/students） 3
2.2 Excel 批量导入算法 3
3. 人脸数据模块详细设计 4
3.1 人脸图片上传与预处理 4
3.2 触发训练判断逻辑 4
4. 考勤任务模块详细设计 5
4.1 考勤任务状态机 5
4.2 识别去重逻辑 5
4.3 考勤结束时补全缺勤记录 6
5. AI 推理模块详细设计 6
5.1 MTCNN 人脸检测参数配置 6
5.2 特征提取关键实现 7
5.3 余弦相似度匹配算法 7
5.4 ModelManager 热加载机制 8
6. 迁移学习训练详细设计 9
6.1 训练配置参数 9
6.2 训练流程伪代码 9
6.3 特征向量构建 10
7. 报表模块详细设计 10
7.1 Excel 导出结构 10
7.2 出勤率统计查询 11
8. 系统配置参数表 11

基于 CNN 的校园课堂智能考勤系统 — 详细设计文档

版本：v1.0

日期：2026-04-16

依据：概要设计文档 v1.1

---

1. 用户认证模块详细设计

1.1 登录接口（POST /api/auth/login）

处理流程：

```
输入：{ username, password }

    1. 参数校验：username/password 非空，长度限制
    2. 查询 users 表：SELECT * FROM users WHERE username=?
    3. 若账号不存在 → 返回 401（不区分账号/密码，防枚举）
    4. 若 is_active=0 → 返回 403
    5. bcrypt.checkpw(password, password_hash) 验证
    6. 失败 → 返回 401
    7. 成功 → 生成 JWT：
           payload = { sub: user.id, role: user.role, exp: now+24h }
           token = jwt.encode(payload, SECRET_KEY, algorithm='HS256')
           返回 { token, user_info }
```

JWT Payload 结构：

```json
{
  "sub": 1,
  "role": "teacher",
  "real_name": " 张三",
  "exp": 1713322800,
  "iat": 1713236400
}
```

路由守卫装饰器实现要点：

```python
# app/utils/auth.py
def require_roles(*roles):
    """ 角色权限装饰器，用法：@require_roles('admin', 'teacher')"""
    def decorator(f):
        @wraps(f)
        def wrapper(*args, **kwargs):
            token = request.headers.get('Authorization', '').replace('Bearer ', '')
            payload = jwt.decode(token, SECRET_KEY, algorithms=['HS256'])
            if payload['role'] not in roles:
                abort(403)
            g.current_user = payload
```

```
            return f(*args, **kwargs)
        return wrapper
    return decorator
```

---

2. 学生档案模块详细设计

2.1 分页查询（GET /api/students）

查询参数:

参数	类型	默认值	说明
page	int	1	页码（从 1 起）
per_page	int	20	每页条数，最大 100
name	str	—	姓名模糊搜索
class_name	str	—	班级精确匹配
face_registered	bool	—	是否已注册人脸

SQL 构建逻辑:

```
query = Student.query
if name:
    query = query.filter(Student.name.ilike(f'%{name}%'))
if class_name:
    query = query.filter_by(class_name=class_name)
if face_registered is not None:
    query = query.filter_by(face_registered=face_registered)
pagination = query.order_by(Student.student_id).paginate(
    page=page, per_page=per_page, error_out=False
)
```

2.2 Excel 批量导入算法

```
读取 Excel 文件 (openpyxl)

    验证表头: [学号, 姓名, 班级, 院系]

    逐行处理:
```

```
        空行跳过
        必填字段校验（学号、姓名、班级）
        学号格式校验（正则：^\d{10}$）
        查询是否已存在 → 已存在则跳过并记录
        构建 Student 对象，加入 session

    批量 session.bulk_save_objects()（提升写入性能）

    返回：{ success: N, failed: M, errors: [...] }
```

---

3. 人脸数据模块详细设计

3.1 人脸图片上传与预处理

```python
# app/services/face_service.py

def upload_and_preprocess(student_id: int, file_storage: FileStorage) -> dict:
    """
    步骤:
    1. 保存原始文件至 dataset/faces/raw/{student_id}/
    2. 调用 FaceDetector.detect(image) 检测人脸
    3. 若未检测到 → 删除原始文件，返回错误
    4. 取置信度最高的人脸 BBox
    5. 裁剪 + Resize 224x224 + 归一化
    6. 保存处理后图片至 dataset/faces/{student_id}/
    7. 写入 face_images 表，is_processed=True
    8. 更新 students.face_count += 1
    9. 若 face_count >= 5: 设置 face_registered=True
    """
```

3.2 触发训练判断逻辑

```
触发条件（满足任一）：
  A. 管理员手动点击"触发训练"
  B. 新注册学生数量 >= 触发批次阈值（默认10人）

训练任务异步执行（避免阻塞 HTTP 请求）：
```

→ 使用 threading.Thread 或 Celery 异步任务
→ 训练完成后更新 ModelManager（热加载新特征库）
→ 训练状态存储于内存或数据库的 training_jobs 表

---

4. 考勤任务模块详细设计

4.1 考勤任务状态机

```
            创建任务


          [PENDING]

      教师点击"开始考勤"


          [RUNNING] ←

      摄像头帧到达 → 识别 → 写入 attendance_records

      教师点击"结束考勤" / 超时自动结束


  标记未出现学生为 absent


          [FINISHED]                          （重新开始？不支持）

  异常：网络断开 → 后端检测 WebSocket 关闭 → 30s后自动转 FINISHED
```

4.2 识别去重逻辑

```python
# 内存中维护已识别集合，任务结束后清理
recognized_students: Dict[int, Set[str]] = {}  # task_id → set(student_id)

def record_if_new(task_id: int, student_id: str, confidence: float):
    if student_id in recognized_students.get(task_id, set()):
```

```
        return  # 已识别，跳过
    recognized_students.setdefault(task_id, set()).add(student_id)
    # 写入 attendance_records
    record = AttendanceRecord(
        task_id=task_id,
        student_id=student_id,
        status='present',
        confidence=confidence,
        recognized_at=datetime.now()
    )
    db.session.add(record)
    db.session.commit()
```

4.3 考勤结束时补全缺勤记录

```
def finalize_task(task_id: int):
    task = AttendanceTask.query.get(task_id)
    enrolled = get_enrolled_student_ids(task.course_id)
    recognized = {r.student_id for r in AttendanceRecord.query
                  .filter_by(task_id=task_id).all()}
    absent = enrolled - recognized
    for sid in absent:
        db.session.add(AttendanceRecord(
            task_id=task_id, student_id=sid,
            status='absent', confidence=0.0
        ))
    task.status = 'finished'
    task.end_time = datetime.now()
    task.present_count = len(recognized)
    task.total_students = len(enrolled)
    db.session.commit()
```

---

5. AI 推理模块详细设计

5.1 MTCNN 人脸检测参数配置

```
# app/ai/face_detector.py
DETECTOR_CONFIG = {
```

```
    "min_face_size": 40,        # 最小人脸尺寸（像素），过滤远距人脸
    "thresholds": [0.6, 0.7, 0.7],  # P-Net, R-Net, O-Net 置信阈值
    "factor": 0.709             # 图像缩放因子
}
```

多人脸处理：一帧图像可能包含多个学生，`detect()` 返回列表，逐个进入特征提取流程。

5.2 特征提取关键实现

```
# app/ai/face_recognizer.py

class FaceRecognizer:
    def __init__(self, model_type: str, weights_path: str, device: str = 'cpu'):
        self.device = torch.device(device)
        self.model = self._build_model(model_type)
        self.model.load_state_dict(torch.load(weights_path, map_location=self.device))
        self.model.eval()

    def _build_model(self, model_type: str) -> nn.Module:
        if model_type == 'vgg16':
            base = torchvision.models.vgg16(pretrained=False)
            # 去掉 classifier，保留 features + avgpool
            return nn.Sequential(base.features, base.avgpool,
                                 nn.Flatten(), nn.Linear(25088, 512))
        elif model_type == 'resnet50':
            base = torchvision.models.resnet50(pretrained=False)
            base.fc = nn.Linear(2048, 512)
            return base

    @torch.no_grad()
    def extract_features(self, face_tensor: Tensor) -> np.ndarray:
        feat = self.model(face_tensor.unsqueeze(0).to(self.device))
        # L2 归一化
        feat = F.normalize(feat, p=2, dim=1)
        return feat.cpu().numpy().squeeze()
```

5.3 余弦相似度匹配算法

```
# app/ai/feature_database.py
```

```python
class FeatureDatabase:
    def __init__(self):
        self.student_ids: List[str] = []
        self.feature_matrix: np.ndarray = None  # shape: (N, 512)

    def query(self, query_vec: np.ndarray, threshold_high=0.75,
              threshold_low=0.5) -> dict:
        """
        返回最相似的学生及其状态
        时间复杂度: O(N) 矩阵乘法, N= 学生数
        百人规模耗时 < 1ms
        """
        if self.feature_matrix is None or len(self.student_ids) == 0:
            return {"status": "unknown"}

        # 余弦相似度 = 点积 (因为已做 L2 归一化, ||a||=||b||=1)
        similarities = np.dot(self.feature_matrix, query_vec)  # shape: (N,)
        best_idx = np.argmax(similarities)
        best_sim = float(similarities[best_idx])

        if best_sim >= threshold_high:
            return {"student_id": self.student_ids[best_idx],
                    "confidence": best_sim, "status": "present"}
        elif best_sim >= threshold_low:
            return {"student_id": self.student_ids[best_idx],
                    "confidence": best_sim, "status": "unverified"}
        else:
            return {"confidence": best_sim, "status": "unknown"}
```

5.4 ModelManager 热加载机制

```python
# app/ai/model_manager.py

class ModelManager:
    _instance = None  # 单例

    @classmethod
    def get_instance(cls):
        if cls._instance is None:
```

```
            cls._instance = ModelManager(current_app.config['AI_CONFIG'])
        return cls._instance

    def reload_feature_db(self):
        """
        训练完成后调用，无需重启服务
        使用读写锁保护并发安全
        """
        with self._lock:
            new_db = FeatureDatabase()
            new_db.load_from_file(self.config['FEATURE_DB_PATH'])
            self.feature_db = new_db
            current_app.logger.info("Feature database reloaded: %d students",
                                    len(new_db.student_ids))
```

---

6. 迁移学习训练详细设计

6.1 训练配置参数

参数	VGG16	ResNet50	说明
输入尺寸	224×224	224×224	与预训练一致
Batch Size	32	32	视显存调整
初始学习率	1e-4	1e-4	微调建议小学习率
冻结层数	features[0:20]	layer1+layer2	保留低层通用特征
Epoch	30	30	含 Early Stopping
优化器	Adam	Adam	
损失函数	CrossEntropy	CrossEntropy	分类阶段
学习率调度	StepLR(step=10, =0.1)	CosineAnnealingLR	
数据增强	RandomHFlip, ColorJitter, RandomRotation(±15°)	同左	防过拟合

6.2 训练流程伪代码

```
初始化：加载 ImageNet 预训练权重
阶段1（冻结底层微调，Epoch 1-10）：
  - 冻结 backbone 前N层
  - 只训练新增分类头（FC layers）
  - 学习率 = 1e-3
```

```
阶段2（全网端到端微调，Epoch 11-30）：
  - 解冻所有层
  - 学习率降至 1e-4
  - 应用 Early Stopping (patience=5)

每个 Epoch 结束：
  - 在验证集上计算 Top-1 准确率
  - 若为最佳 → 保存 checkpoint

训练完成后：
  - 在测试集上评估（准确率、F1、AUC）
  - 提取所有注册学生特征向量 → 保存 features.npy
  - 调用 ModelManager.reload_feature_db()
```

6.3 特征向量构建

```python
def extract_all_features(model, dataloader, student_ids):
    """
    遍历所有学生的人脸图片，对每个学生取多张图片特征的均值向量。
    均值特征比单张更鲁棒（表情、角度变化的平均表示）。
    """
    feature_dict = {}
    for student_id in student_ids:
        imgs = load_student_images(student_id)  # 取所有图片
        feats = [model.extract_features(img) for img in imgs]
        avg_feat = np.mean(feats, axis=0)
        avg_feat = avg_feat / np.linalg.norm(avg_feat)  # 再次归一化
        feature_dict[student_id] = avg_feat
    return feature_dict
```

---

7. 报表模块详细设计

7.1 Excel 导出结构

```
Excel 文件结构（openpyxl）：
  Sheet 1: 考勤汇总
```

列：学号 | 姓名 | 班级 | 出勤次数 | 缺勤次数 | 待核实次数 | 出勤率

Sheet 2：考勤明细
列：课程名 | 考勤日期 | 学号 | 姓名 | 状态 | 置信度 | 识别时间 | 核实操作员

Sheet 3：缺勤预警
列：学号 | 姓名 | 班级 | 连续缺勤次数 | 最近缺勤日期

7.2 出勤率统计查询

```
-- 按课程统计每次考勤出勤率（用于折线图）
SELECT
    t.task_date,
    t.id AS task_id,
    t.total_students,
    t.present_count,
    ROUND(t.present_count / t.total_students * 100, 2) AS attendance_rate
FROM attendance_tasks t
WHERE t.course_id = :course_id
  AND t.status = 'finished'
  AND t.task_date BETWEEN :start_date AND :end_date
ORDER BY t.task_date;
```

---

8. 系统配置参数表

所有运行时可调参数集中在 `app/config.py`：

配置键	默认值	说明
SIMILARITY_THRESHOLD_HIGH	0.75	识别成功阈值
SIMILARITY_THRESHOLD_LOW	0.50	待核实阈值
ABSENCE_TRIGGER_COUNT	3	连续缺勤 N 次触发预警
FACE_MIN_COUNT	5	人脸注册最少图片数
TRAIN_TRIGGER_BATCH	10	新注册 N 人后自动触发训练
WS_FRAME_INTERVAL_MS	500	WebSocket 帧推送间隔（前端参考）
MODEL_TYPE	resnet50	当前使用的 CNN 骨干网络
DEVICE	cpu	推理设备，可设为 cuda:0
MAX_FACE_IMAGE_SIZE_MB	5	上传图片大小限制

配置键	默认值	说明
`JWT_EXPIRY_HOURS`	24	JWT Token 有效期

文档版本：*v1.0* | 最后更新：*2026-04-16*